
ISyE 6740 - Spring 2021
Project Proposal or Final Report

Team Member Names: Bakiye Hilal Khaniyev, Yussif Salama & Aswath Oruganti
Project Title: Open-Set Recognition of Plant Species: Detecting Unfamiliar Flora for Biodiversity Screening

1 Problem Statement

This report details the progress made toward developing a robust classification and open-set recognition system for plant species identification. The primary objective is to build a system capable of accurately classifying known plant species while successfully flagging unfamiliar taxa as “unknown,” a critical requirement for integrating field data into larger ecological tracking databases. To date, the data curation pipeline has been finalized, and a baseline ResNet-18 model has been established. We have successfully fine-tuned the model for 5 epochs on the “Known” species subset and conducted a comparative analysis of the latent embedding space (Pre-trained vs. Fine-tuned) using Principal Component Analysis (PCA) and Mutual Information (MI) metrics.

2 Data Source

A robust data curation pipeline was implemented via the `PlantNetDataCurator` class and custom synchronization scripts to address the severe class imbalances inherent in the PlantNet-300K dataset.

1. Metadata Curation & Disk Syncing

The dataset is initially filtered using the `PlantNetDataCurator` class, which parses the JSON metadata to enforce strict support boundaries (e.g., minimum 150 images, maximum 300 images per species). This prevents dominant classes from overwhelming the gradient during training. However, because metadata does not always match local disk contents, we implemented a physical sync function:

Listing 1: Disk Syncing Logic

```
1 def sync_dataframe_with_disk(df, base_image_dir, max_per_class=300):
2     # Iterates through the curated dataframe and verifies the physical .jpg exists
3     # If it exists, it replaces the metadata ID with the actual_filename
4     ...
5     # Re-balancing logic caps only the 'train' split at max_per_class
6     n_samples = min(len(train_df), max_per_class)
7     balanced_chunks.append(train_df.sample(n=n_samples, random_state=42))
```

Why this matters: This code ensures our PyTorch `DataLoader` never crashes due to a `FileNotFoundError`, and guarantees our training data remains perfectly balanced across all known classes.

Once curated, the data is strictly divided into two pools:

- **Known Species (80%):** Used for standard training and validation.
- **Unknown Species (20%):** Strictly held out during training to simulate true “novel” encounters during the evaluation phase.

3 Methodology

The project utilizes a deep Convolutional Neural Network (CNN) to extract high-dimensional representations of the botanical images.

1. ResNet-18 and Feature Hooks

Instead of just grabbing the final classification prediction, we need the deep mathematical representation (embeddings) of the image to perform Open-Set Recognition. We achieve this by subclassing ResNet-18 and attaching a “hook” to the pooling layer:

Listing 2: ResNet-18 with Forward Hook

```

1 class OpenSetResNet(nn.Module):
2     def __init__(self, num_known_classes):
3         super(OpenSetResNet, self).__init__()
4         self.backbone = models.resnet18(weights=models.ResNet18_Weights.
5             IMAGENET1K_V1)
6         in_features = self.backbone.fc.in_features
7         self.backbone.fc = nn.Linear(in_features, num_known_classes)
8         self.embeddings = None
9         self._register_hook()
10
11     def _register_hook(self):
12         # Captures the 512-D vector just before the final classification layer
13         def hook(module, input, output):
14             self.embeddings = output.squeeze()
15             self.backbone.avgpool.register_forward_hook(hook)

```

Why this matters: The `register_forward_hook` allows us to intercept the 512-dimensional vector that represents the “essence” of the plant. We use these vectors for our PCA and Mutual Information analysis.

2. Checkpointing

To optimize our workflow, we implemented a save/load checkpointing system in the execution block:

Listing 3: Checkpoint Save and Load

```

1 weights_path = 'resnet18_finetuned_knowns.pth'
2 if os.path.exists(weights_path):
3     pretrained_model.load_state_dict(torch.load(weights_path, map_location=device)
4         )
5     pretrained_model.eval()
6 else:
7     trained_model = train_model(...)
8     torch.save(trained_model.state_dict(), weights_path)

```

Why this matters: This prevents the need to retrain the 5-epoch model every time we want to adjust our visualization or analysis code, saving significant compute time.

4 Results & Latent Space Analysis

To evaluate whether the model is learning to distinguish between “Known” and “Unknown” species without explicit supervision on the unknowns, we extracted and analyzed the 512-D embeddings of a mixed test set containing both pools.

1. Principal Component Analysis (PCA) Projection

PCA was utilized to compress the 512-dimensional embeddings into a 2D space for visual inspection of class separation.

Pre-Trained Baseline: Using raw ImageNet weights, the PCA projection showed significant overlap. The model relied on generic visual features (e.g., basic shapes, greens/browns), which were insufficient for separating familiar flora from novel species.

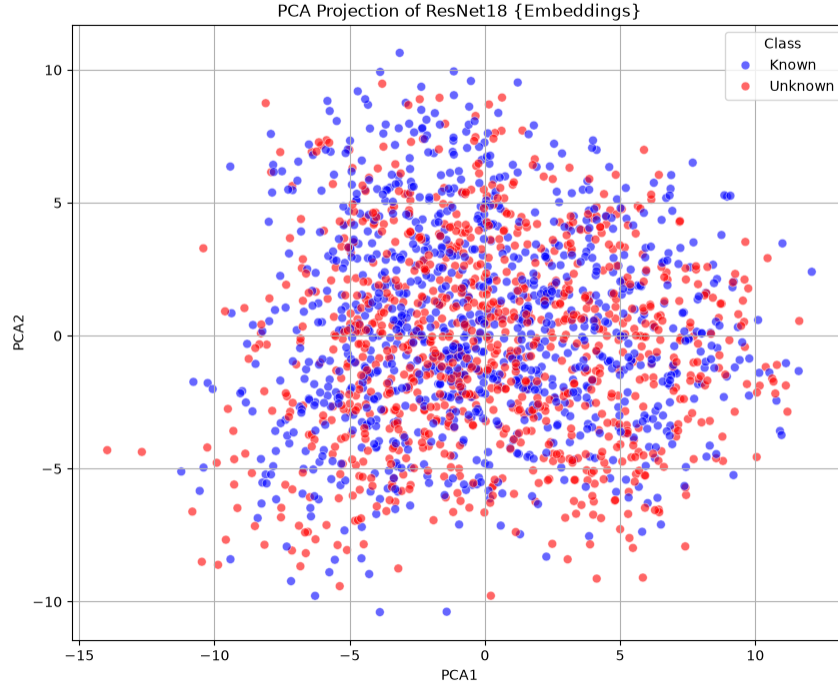


Figure 1: PCA Projection of Embeddings - Pre-Trained ResNet-18 Baseline

After 5 Epochs: The fine-tuned embeddings began to cluster more distinctly based on the specific morphological traits of the known classes. While the decision boundary is not perfectly linear, the manifold topology indicates the model is actively learning domain-specific botanical features.

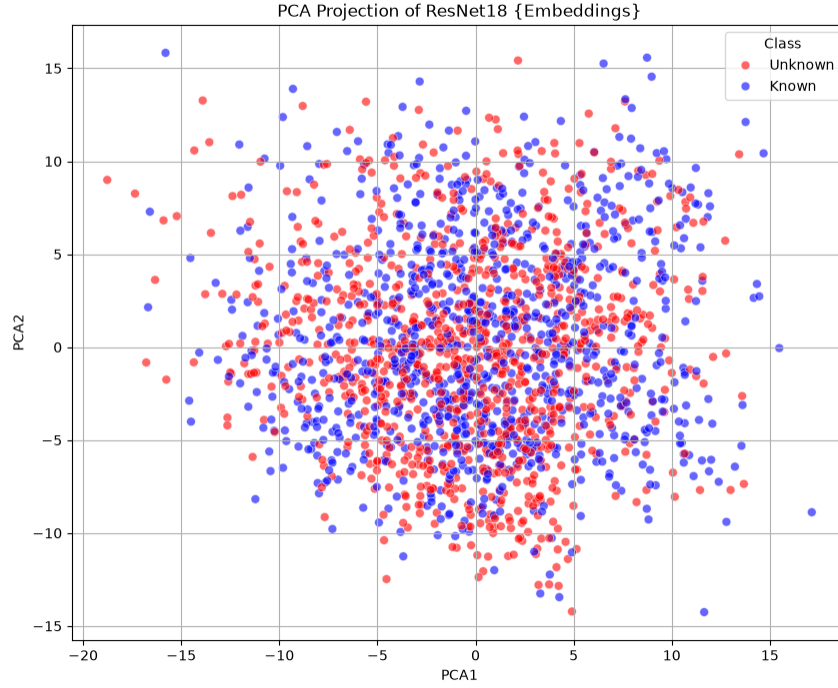


Figure 2: PCA Projection of Embeddings - After 5 Epochs of Fine-Tuning

2. Mutual Information (MI) Scores

We calculated the Mutual Information between each of the 512 embedding dimensions and the binary target of “Familiarity” (1 for Known, 0 for Unknown).

Pre-Trained Baseline: The MI scores were relatively low and evenly distributed, indicating no specific feature maps were tuned to detect botanical novelty.

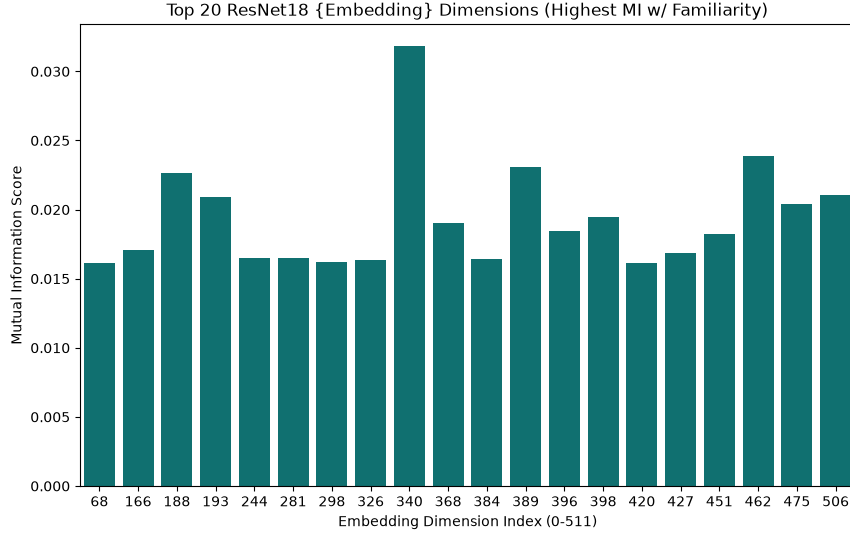


Figure 3: Mutual Information Scores - Pre-Trained ResNet-18 Baseline

After 5 Epochs: A sharp hierarchy emerged in the MI scores. Specific dimensions within the 512-D vector developed highly significant correlations with the Known/Unknown label. This proves the fine-tuning process forced certain neural pathways to become highly specialized in recognizing the specific characteristics of our curated dataset.

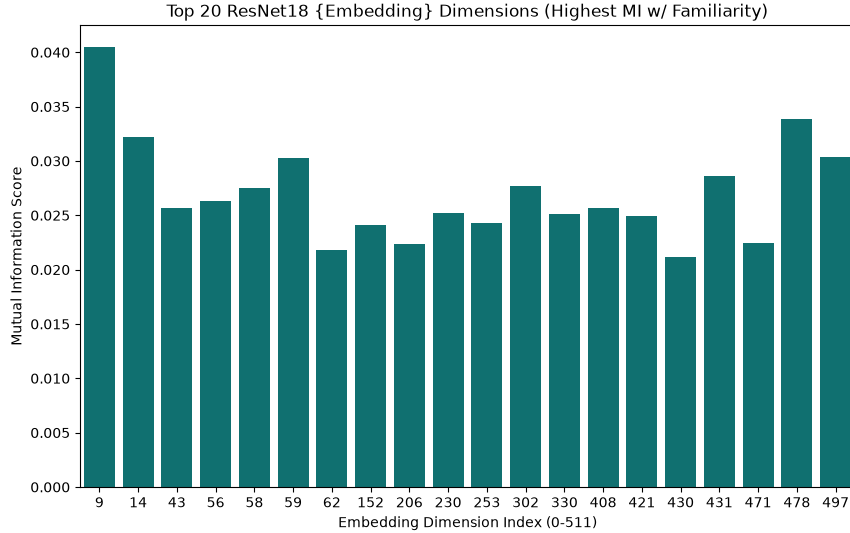


Figure 4: Mutual Information Scores - After 5 Epochs of Fine-Tuning

3. Activation Fingerprints

We reshaped the 512-D vectors into 16x32 grids to visualize the activation intensity of the neural network.

Pre-Trained Baseline: Activations were diffuse and “fuzzy.”

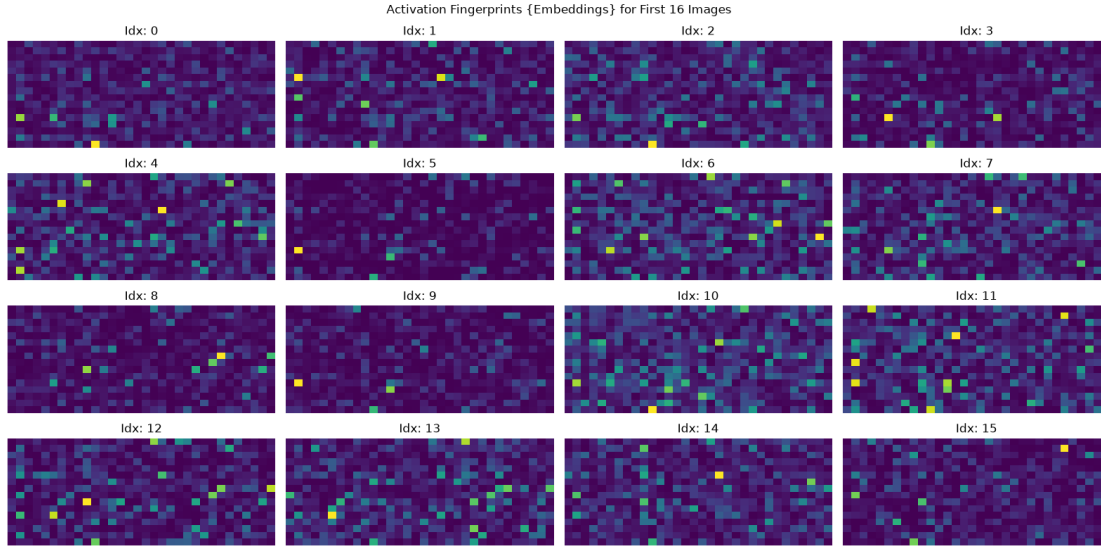


Figure 5: Activation Fingerprints - Pre-Trained ResNet-18 Baseline

After 5 Epochs: The 5-epoch model displays much higher contrast and sparser, more decisive activations. This confirms the network is making more confident and specific feature extractions rather than relying on generalized ImageNet patterns.

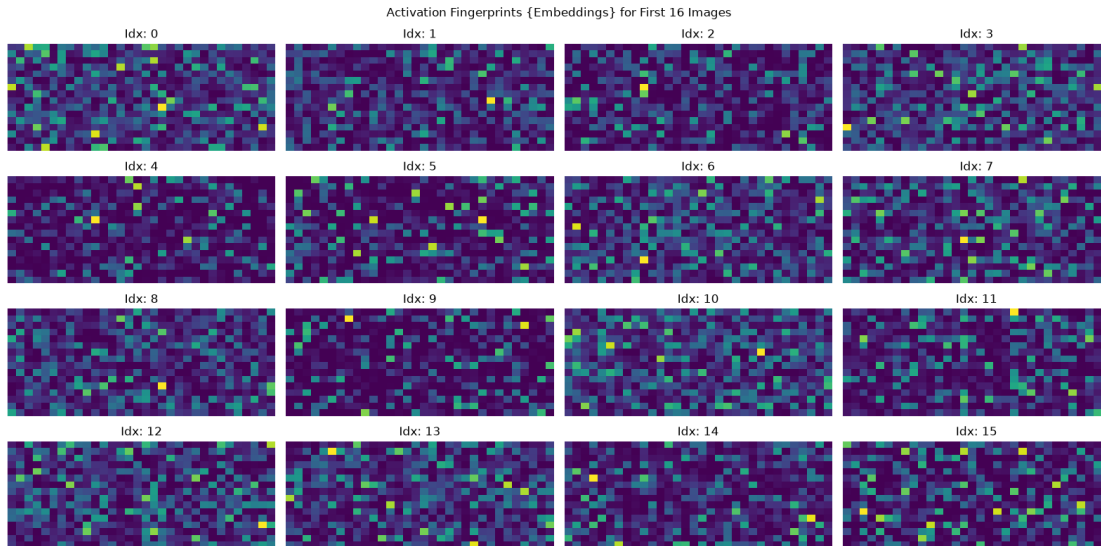


Figure 6: Activation Fingerprints - After 5 Epochs of Fine-Tuning

5 Next Steps & Project Roadmap

The foundation for the open-set recognition pipeline is fully functional. To complete the objectives outlined in the project proposal, the following steps will be executed next:

1. **Algorithmic Thresholding:** We will implement statistical methods (e.g., Softmax thresholding,

OpenMax, or Mahalanobis distance) on top of the current embeddings to establish a definitive quantitative threshold for rejecting an image as “Unknown.”

2. **Extended Training:** The current model was evaluated after 5 epochs. Extending training with a dynamic learning rate scheduler is expected to further tighten the PCA clusters and improve overall accuracy.
3. **Architecture Expansion (Optional):** Integrating **EfficientNet-B0** alongside ResNet-18 will allow us to compare computational efficiency versus accuracy, which is highly relevant for integrating these models into real-world database tracking applications.